

Phase 3 — 数据分析 Logic 层

SPEC-007 | Status: 设计中 | 依赖: SPEC-003 (数据库/Redis), SPEC-004 (Skill Context/LLM Router), SPEC-006 (知识库基础设施, Phase 3)

目标

实现数据分析核心 Logic 层, 为 Skill 提供可复用、可独立测试的数据处理与校验能力。本 spec 不包含任何 Skill 接口实现, 只输出可被 Skill 调用的函数/服务。

Skill 包装层见 **SPEC-008 (Skill 实现层)**。Logic 层必须先于 Skill 层完成, Skill 仅做参数校验、权限检查、调用 Logic、格式化输出。

前置依赖检查

前置 Spec	状态	备注
SPEC-002	✅/❌	CI Pipeline 就绪
SPEC-003	✅/❌	MongoDB / Redis 可用
SPEC-004	✅/❌	SkillContext 注入机制可用
SPEC-006	✅/❌	GridFS + Milvus Collection 可用 (knowledge Logic 依赖)

背景

数据分析的核心算法不应和 Skill 接口耦合: - SQL 生成与安全校验可被 `sql_executor` Skill 和 `save_analysis_report` Skill 复用 - 统计算法需要独立单元测试 (输入固定数据 → 输出确定结果) - 报告校验逻辑需要被 Agent 在重试循环中反复调用

因此将数据分析 Logic 抽出为独立 spec, 在 Phase 2 实现 SQL/Stats/Report 基础部分, Phase 3 补充 Knowledge Search Logic。

详细设计

1. SQL Logic (`internal/logic/sql/`)

1.1 SQL 安全校验

- 基于 `pingcap/tidb/parser` 解析 AST
- 仅允许: `SELECT` / `DESCRIBE` / `SHOW` / `EXPLAIN`
- 禁止: DML/DDL、子查询超过 2 层、无 WHERE 的 DELETE/UPDATE (即使 DML 被禁止也双重拦截)
- 返回: `{allowed: bool, reason: string}`

1.2 SQL 执行与缓存

- 参数化查询: 禁止字符串拼接用户输入
- 结果缓存: Redis Key `sql:cache:{sha256(sql+params)}`, TTL 5min
- 错误友好提示:
- 表不存在 → 返回该用户有权限访问的表名列表
- 字段错误 → 返回表结构 (字段名 + 类型)

1.3 自然语言 → SQL 生成 (可选 Phase 3 增强)

- 输入: 用户问题 + 数据库 Schema (表/字段/关系)
- 输出: 参数化 SQL + 参数列表
- 校验: 生成后再次走 AST 安全校验

2. Stats Logic (`internal/logic/stats/`)

2.1 基础统计 (Phase 2)

- 线性回归 / 多元回归 (gonum)
- 时间序列分解: 趋势项 + 季节项 + 残差
- 描述性统计: 均值/中位数/方差/分位数

2.2 高级统计 (Phase 4)

- 聚类分析: K-Means (gonum)
- 主成分分析: PCA (gonum)
- 财务分析: 常用比率 (流动比率、ROE、毛利率) + 同比/环比

2.3 输出规范

所有 Stats Logic 函数返回统一结构:

```
{
  "method": "linear_regression",
  "parameters": {"x_col": "age", "y_col": "income"},
  "result": {},
  "summary": "...",
  "warnings": ["样本量 < 30, 结果可能不稳定"]
}
```

- 数值安全: NaN / Inf 检测 → 替换为 `null` + 友好提示

3. Knowledge Search Logic (`internal/logic/knowledge/`)

3.1 混合搜索 (Phase 3)

- 向量搜索: Milvus `similarity_search`, 返回 top-50
- 全文搜索: MongoDB `$text` 索引, 返回 top-50
- RRF 融合排序: Reciprocal Rank Fusion, k=60, 取 top-5

3.2 权限与隔离

- 权限过滤: 按用户角色裁剪搜索结果
- doc_id 隔离: 跨文档不串数据
- 上下文截断: top-5 × 800 tokens (由 SPEC-004 上下文管理调用)

4. Report Validation Logic (`internal/logic/report/`)

4.1 Markdown AST 解析

- 提取标题层级 (H1-H3)
- 提取章节段落位置

4.2 必含章节检查

- 摘要
- 数据来源
- 分析方法
- 关键指标
- 结论

4.3 修正反馈生成

- 校验失败 → 返回 `feedback` 字符串, 指出缺失章节和示例写法

- 由 Agent 重试循环使用（最多 3 次）
- 校验记录写入 `report_validation_logs`

与 Skill 层的边界



可行性分析

检查项	结论
是否需要新 DB 集合	Yes (report_validation_logs)
是否影响现有 API	No (Logic 层不直接暴露 HTTP API)
性能影响	SQL 缓存命中 < 5ms; KB Search < 200ms; Stats 计算 < 1s
是否需要新增 Skill	No (Skill 在 SPEC-008 实现)
是否需要 E2E 测试	No (以单元测试为主)

相关文件

文件	角色	变更幅度
<code>internal/logic/sql/validator.go</code>	SQL AST 安全校验	新建
<code>internal/logic/sql/executor.go</code>	SQL 执行与缓存	新建
<code>internal/logic/stats/regression.go</code>	回归分析	新建
<code>internal/logic/stats/timeseries.go</code>	时间序列	新建
<code>internal/logic/stats/clustering.go</code>	聚类分析 (Phase 4)	新建
<code>internal/logic/stats/pca.go</code>	PCA (Phase 4)	新建
<code>internal/logic/stats/financial.go</code>	财务分析 (Phase 4)	新建
<code>internal/logic/knowledge/search.go</code>	混合搜索 (Phase 3)	新建
<code>internal/logic/report/validator.go</code>	报告格式校验	新建
<code>internal/logic/openapi/parser.go</code>	OpenAPI 解析 + MCP Tool 生成	新建 (Phase 4)

验证标准

- 1. `SQLValidator("SELECT * FROM users")` → 允许
- 2. `SQLValidator("DROP TABLE users")` → 拒绝, 返回明确原因
- 3. Stats 输入固定数据 → 输出确定的回归系数/聚类标签
- 4. 报告缺失「结论」章节 → `ReportValidator()` 返回缺失章节 feedback
- 5. Logic 函数独立运行: 不依赖 Skill 注册、不依赖 HTTP 上下文